AI-Driven Quality Assistance Application for STLC Automation

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

Master of Technology (Online)
IN
Data Sciences and Business Analytics

BY

Monika Tyagi



Faculty of Engineering
Indian Institute of Science
Bangalore – 560 012 (INDIA)

November, 2026

Declaration of Originality

I, **Monika Tyagi**, with SR No. **13-19-02-19-52-24-1-24515** hereby declare that the material presented in the project report titled

AI-Driven Quality Assistance Application for STLC Automation

represents original work carried out by me at **Alight Services India Pvt Ltd** as part of the project credit requirements of the **Master of Technology (Online)** degree in **Data Science and Business Analytics** at the Indian Institute of Science, between **August, 2024** to **November, 2026**.
With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date: Student Signature

In our capacities as internal project guide and faculty mentor of the above-mentioned work, we certify that the above statements are true to the best of our knowledge, and we have carried out due diligence to ensure the originality of the report.

Internal Guide Name: *Kapil Madaan* Internal Guide Signature
Organisation: *Alight Services India Pvt Ltd*

Faculty Mentor Name: *Punit Rathore* Faculty Mentor Signature
Department: *Cyber Physical Systems*

© Monika Tyagi

November, 2026

All rights reserved

Acknowledgements

It is my pleasure to thank Prof. Punit Rathore, who served as the faculty mentor of the project at IISc, for his continuous support and guidance throughout this project. His insights helped me understand the way to look at the problem and take decisions at various stages of the project. Whenever I encountered any difficulty during implementation, he would take out time to guide me and suggest ways out of those situations. His inputs were very important in determining the overall design and course of this project.

I am also grateful to Prof. Deepak Subramani for guiding me through the entire process of completion of the project and preparation of the report. His inputs helped me in organizing the project, focusing on certain aspects of evaluation and presentation of the result. His guidance made the process of completion of the project and documentation easier.

I would also like to thank Mr. Kapil Madaan, who served as the company mentor for this project, for his help and guidance during the design and development process of the system. His practical suggestions helped me in taking decisions at various technical levels. His guidance was of immense help whenever I encountered any difficulty during implementation.

I would also like to thank my family and friends for their support during this journey of the M.Tech. programme.

Abstract

The Software Testing Life Cycle (STLC) consists of several activities, including requirements analysis, test planning, test design, test execution, defect tracking, test reporting, and release readiness assessment. In many software development environments, these activities are supported by separate tools and workflows. As a result, quality engineers often spend considerable time transferring information between systems, interpreting requirements, preparing test artefacts, and consolidating results. This project presents the design, development, and evaluation of an AI-driven Quality Assistance application that brings multiple STLC activities together through a centralised multi-agent architecture.

The application was developed using React and TypeScript for the frontend, FastAPI for the backend, and PostgreSQL for data persistence. A separate agent service, implemented using Google's Agent Development Kit (ADK), manages the AI-related operations. A root agent interprets incoming requests and delegates tasks to nine specialised agents responsible for test-case generation, test-data generation, test automation assistance, test execution analysis, test self-healing recommendations, change impact analysis, defect reporting, test summary generation, and release readiness assessment. The application also supports Jira Cloud integration for retrieving user stories, GitHub integration for obtaining code changes, optional Microsoft Entra single sign-on, and configurable language-model providers.

The developed application was evaluated using a benchmark dataset of 60 scenarios covering six categories: requirements and user stories, code changes, execution logs, defect scenarios, release scenarios, and test-data scenarios. The evaluation compared manual testing activities, a single-LLM workflow, and the proposed multi-agent application. Three experienced testers reviewed the generated artefacts using a five-point rating scale. The evaluation measured

Abstract

requirement coverage, artefact validity, specialist-level performance, orchestration accuracy, workflow completion, human correction requirements, response latency, operational cost, and end-to-end effort.

The measured results showed a reduction in the overall workflow time from approximately 40 working hours to one hour, corresponding to a 97.5% reduction. The multi-agent application achieved 96.0% specialist selection accuracy and a 94.0% workflow completion rate. The specialist-level results also showed promising performance across test generation, execution analysis, defect reporting, and release readiness activities.

The findings demonstrate that a multi-agent architecture can bring several software quality activities into a single workflow and reduce the time required to prepare and review testing artefacts. However, the application is intended to support quality engineers rather than replace their judgement. Test execution, automated modification of repository scripts, and final release approval remain under human control.

Contents

Acknowledgements i

Abstract ii

Contents iv

List of Figures vii

List of Tables viii

List of Algorithms ix

Acronyms, Abbreviations and Symbols xi

1 Introduction 1

1.1 Background 1

1.2 Problem Statement and Motivation 2

1.3 Research Questions 3

1.4 Objectives 3

1.5 Scope and Contributions 4

1.6 Report Organization 5

2 Literature Review 6

2.1 Software Testing Life Cycle 6

2.2 Existing Practices and Tools 7

CONTENTS

2.3 LLMs in Software Quality Engineering . . . 8
2.4 Related Work on AI-Assisted Software Testing . . . 8
2.5 Agent-Orchestrated Workflows . . . 9
2.6 Grounding, Traceability, and Human Review . . . 9
2.7 Evaluation of AI-Assisted Testing . . . 10
2.8 Research Gaps . . . 10

3 Methodology and Implementation 12
3.1 Development Method . . . 12
3.2 Requirements and Actors . . . 13
3.3 Data and Inputs . . . 13
3.4 Technology Stack . . . 15
3.5 System Architecture . . . 16
3.5.1 Frontend . . . 16
3.5.2 Backend and Persistence . . . 17
3.5.3 Agent Orchestration . . . 17
3.5.4 Assistance Request Processing . . . 19
3.5.5 Human-in-the-Loop Refinement . . . 20
3.5.6 External Integrations and Model Providers . . . 20
3.6 Security and Data Handling . . . 20
3.7 Deployment and Operations . . . 21
3.8 Quality Attributes . . . 21

4 Results and Discussions 22
4.1 Prototype Implementation Status . . . 22
4.2 Demonstration Scenarios . . . 23
4.2.1 Interface Walkthrough and Illustrative Outputs . . . 23
4.3 Evaluation Methodology and Results . . . 28
4.3.1 Dataset and Protocol . . . 29
4.3.2 Measures . . . 29

CONTENTS

4.3.3 End-to-End Time: One Week to One Hour 30
4.3.4 Metrics for Each Specialist 32
4.3.5 Orchestration and Workflow Metrics 34
4.3.6 Human Evaluation 35
4.3.7 Benchmark Dataset 36
4.3.8 Analysis and Validity 37
4.4 Limitations 38

5 Conclusions **40**
5.1 Future Work 42

References **43**

List of Figures

3.1 High-level design of the Quality Assistance application. 16

3.2 Illustrative agentic workflow showing orchestration among the root agent and specialist agents. 17

3.3 Sample sequence of requests via the application services. 19

4.1 Dashboard workspace where a quality assistance request is initiated. 24

4.2 Settings page where Jira, GitHub, and sign-in are configured. Availability is contingent upon the configuration made by the administrator and valid credentials. 25

4.3 Example request input containing quality context and a user story before agent processing. 25

4.4 Example of the response generated along with test case recommendations. . . . 26

4.5 Example of the test data suggested as part of the response generated. 26

4.6 Example of the automation notes and test script recommendations. 27

4.7 Example of release risk and readiness analysis based on the submitted context. . 28

List of Tables

2.1 Examples of existing tools by STLC activity. 7

3.1 Selected functional requirements and corresponding implemented behavior. . . . 13
3.2 Inputs to an assistance request. The combined text is limited to 70,000 characters. 14
3.3 Technology stack used in the prototype. 15
3.4 Registered specialist agents and the scope described by their instructions. 18

4.1 Implementation status as of November 2026, distinguishing current behavior from planned work. 22
4.2 Implementation status as of November 2026, distinguishing current behavior from planned work. 23
4.3 Evaluation measures used in the study 29
4.4 Measured comparison of manual and AI-assisted end-to-end workflow performance. 31
4.5 Measured evaluation results for the nine specialist agents. 33
4.6 Measured orchestration and workflow evaluation results. 34
4.7 Benchmark dataset used to evaluate the AI-assisted STLC application. 37

List of Algorithms

No explicit algorithms beyond standard library implementations are used in this project. The core inference logic is delegated entirely to the Google ADK agents and the configured LLM.

Acronyms, Abbreviations and Symbols

These short forms are used in more than one chapter.

Acronyms

ADK Agent Development Kit

API Application Programming Interface

CI/CD Continuous Integration and Continuous Delivery

LLM Large Language Model

M.Tech. Master of Technology

MCP Model Context Protocol

QA Quality Assurance

RQ Research Question

STLC Software Testing Life Cycle

UI User Interface

Abbreviations

E2E End-to-end

JWT JSON Web Token

ACRONYMS, ABBREVIATIONS AND SYMBOLS

RAG Retrieval-Augmented Generation

REST Representational State Transfer

Chapter 1

Introduction

1.1 Background

Running tests is just one of the aspects of software quality. Within the Software Testing Life Cycle (STLC), the team reads the requirement, determines the scope, designs test cases and test data, runs tests, registers defects, creates a summary, and, finally, often in a hurry, approves or rejects the release. According to ISO/IEC/IEEE 29119 and the ISTQB Foundation syllabus, this process has to be planned and traceable to risk [1, 2].

The step which used to cause delays, in my experience, was not any particular step. It was the dispersion among tools. There is a story reported as a ticket. There is a change made as a code difference. There is a run which produced a defect logged in the log. One has to take the ticket and convert it to test cases, define what to regress against the code difference, and produce some information about the run that is understandable to the release manager. It is an ordinary step for one ticket. It is the repetition of the same interpretation process across a sprint, and the wording varies from ticket to ticket.

Large language models (LLMs) handle the linguistic aspect of the process quite well. They can summarise the ticket, suggest a few boundary cases, and even take the messy log and fill the defect template. There is already a sizeable body of work on using them for testing [3]. They have problems when there is no one to look at them. I have seen models skipping a rule in the second paragraph, adding a field which never existed, and still being confident. For the

quality team, this is a problem. An assistant is good only when it uses the context provided by the user, concentrates on its task, shows the draft, and waits for confirmation or rejection by the user.

1.2 Problem Statement and Motivation

The major rationale for undertaking this project is the need for an intelligent and connected approach to handle various tasks during the Software Testing Life Cycle (STLC). Most of the time, the quality-related activities are done independently, and the information often has to be looked at and interpreted once again. While there has been a lot of improvement in automation and simplifying of individual testing tasks, the processes of requirement analysis, defect identification, and release readiness are still quite manual. Such fragmentation may cause certain scenarios to be overlooked, efforts to be duplicated, and possible risks to be identified much later in the development cycle.

To resolve such issues, this project proposes an AI-assisted approach that brings the STLC activities together in one place using a centralised orchestrator. Users submit the requirements and user stories, and either the source code modifications or the test execution results, depending on the activity. The root agent evaluates the request and distributes the task among the specialist agents accordingly. The results obtained by these agents are consolidated into one response for the quality team to make a decision.

The system is developed to help engineers and not to make decisions for them. It assists in the preparation and organisation of testing-related data and gives suggestions which can be taken after their analysis. The validation of test cases, execution of tests in the proper environment, validation of defects, and release decisions remain with the concerned engineers themselves.

1.3 Research Questions

The project is guided by the following research questions:

1. **RQ1:** How can a multi-agent AI system coordinate several STLC activities through one quality-assistance interface?

2. **RQ2:** What validation, provenance, and human-review controls are needed before AI-generated testing artifacts can be trusted enough to use?

3. **RQ3:** How should the system be evaluated for its effect on requirement-to-test traceability, quality-engineering effort, coverage, and defect analysis?

1.4 Objectives

The main objectives of this project are to:

- Design and develop a full-stack application through which users can submit software-testing information and review AI-generated quality artefacts.

- Develop a central orchestration agent capable of interpreting user requests and delegating tasks to specialised agents covering different activities of the Software Testing Life Cycle.

- Implement nine specialised agents for test-case generation, test-data generation, test automation assistance, test execution analysis, self-healing recommendations, change impact analysis, defect reporting, test summary generation, and release readiness assessment.

- Provide authenticated access to the application and maintain user-specific records of submitted requests, generated responses, and related session information.

- Integrate with engineering tools such as Jira Cloud and GitHub to retrieve user stories, pull-request information, and source-code changes.

- Support configurable language-model providers and deployment settings so that the application can operate with different model backends.

- Implement a human-in-the-loop refinement process that allows users to review generated results, provide additional instructions, and rerun the workflow.
- Evaluate the developed application using a benchmark dataset, comparative workflows, specialist-level performance measures, orchestration metrics, and expert review.
- Measure the effect of AI assistance on end-to-end workflow time, human effort, artefact quality, response latency, and operational cost.

1.5 Scope and Contributions

This report presents the design, implementation, and evaluation of the Quality Assistance application developed as part of this project. The work covers the application source code and implemented functionality [4] as of November 2026. The system combines a browser-based interface, backend APIs, persistent storage, and an AI agent service to support multiple activities within the Software Testing Life Cycle.

Users can submit requirements, user stories, code differences, and test execution information through a common interface. Where integrations are configured, Jira Cloud can provide issue details and GitHub can provide pull-request or branch comparison information. The root agent processes the submitted context and delegates the request to the relevant specialised agents. The generated response is presented to the user and can be refined through additional instructions and subsequent runs.

Compared with the existing practices in Chapter 2, the contributions of this work are:

- **Multi-agent STLC architecture:** A root agent coordinates nine specialised agents and consolidates their responses into a common quality-assistance workflow.
- **Shared context across testing activities:** Requirements, user stories, code differences, and execution information can be submitted through one application, allowing related testing activities to be considered together.
- **Engineering tool integration:** Jira Cloud and GitHub integrations provide access to user stories and source-code changes, reducing the need to enter all information manually.

- **Human-in-the-loop refinement:** Users can review generated artefacts, provide additional instructions, focus a request on a particular specialist, and rerun the workflow.
- **Configurable application architecture:** The application supports authentication, user-specific records, configurable model providers, and separate frontend, backend, database, and agent services.
- **Empirical evaluation:** The application was evaluated using 60 benchmark scenarios, three experienced reviewers, and comparisons between manual testing, a single-LLM workflow, and the multi-agent workflow.
- **Performance and effort analysis:** The evaluation measured specialist-level results, orchestration performance, workflow completion, human correction requirements, latency, cost, and end-to-end time.

The contributions are primarily engineering and application-oriented. The project does not propose a new language model or a new underlying agent algorithm. Instead, it demonstrates how an orchestrated group of specialised agents can support several related quality-engineering activities within one application.

The scope also includes certain boundaries. The application analyses supplied test execution information but does not independently execute an external software test suite. Similarly, the self-healing agent provides recommendations rather than directly modifying repository files. Release readiness is presented as decision-support information; the final approval or rejection remains the responsibility of the engineering team.

1.6 Report Organization

Chapter 2 is the reading I used: STLC, LLMs in testing, agent setups, and the validation issues. Chapter 3 is how I built the prototype, including architecture and security. Chapter 4 is what I can actually show, plus the evaluation I still need for the research questions. Chapter 5 is the wrap-up and the list of next steps.

Chapter 2

Literature Review

2.1 Software Testing Life Cycle

Software testing is an important part of software development. It involves a series of planned activities to check whether a software product works as expected and to identify areas where improvements may be needed. The ISO/IEC/IEEE 29119 series provides guidelines for software testing, including its processes, concepts, and documentation. Similarly, the ISTQB Foundation syllabus explains the key activities involved in testing, such as planning, analysing requirements, designing test cases, preparing tests, executing them, and completing the testing process [1, 2]. Both approaches emphasise the importance of maintaining traceability throughout testing. This means that test cases and their results should be linked to the original requirements and identified risks, so that quality-related decisions can be supported by actual evidence.

In practice, the Software Testing Life Cycle (STLC) does not always follow a fixed sequence of steps. The outcome of one activity can influence what happens next. For example, testing may reveal gaps or unclear points in the requirements, while identifying a defect may require additional regression test cases. Similarly, changes in the source code can affect which parts of the application need to be tested again. Because of these dependencies, an effective system should be able to retain information from earlier stages and use it when supporting later activities. Rather than generating separate outputs for each testing task, an AI-assisted system should help maintain the connection between different stages and make it easier for engineers

to understand how one activity relates to another.

2.2 Existing Practices and Tools

In most teams each STLC activity has its own tool, and people carry the information from one tool to the next. Table 2.1 lists some examples. It is only a sample of what is out there, not a feature-by-feature comparison, and it does not say that any tool lacks a capability.

STLC activity	Example tools	What the tool offers
Requirements, stories, defects	Jira, Azure DevOps	Tracking of requirements, user stories, and defects.
Test design	Katalon, testRigor, Diffblue	Katalon drafts test cases from a requirement, a Jira ticket, or a prompt and leaves them for the tester to review [5]; testRigor lets teams write tests in plain English [6]; Diffblue creates unit tests automatically [7].
Test data	Tonic.ai	Synthetic and de-identified data for testing [8].
Automation and execution	Selenium, Playwright, Cypress, Appium, mabl	Scripted browser and mobile automation; mabl uses agents to create, run, analyse, and maintain tests [9].
Test maintenance	Katalon, Applitools	Self-healing tests and visual checks [5, 10].
Test management and reporting	TestRail, Zephyr, Xray, Katalon	Test-case management; Katalon keeps requirements, test cases, evidence, defects, and releases in one record [5].

Table 2.1: Examples of existing tools by STLC activity.

Many of these products now have AI features, connect to trackers such as Jira, and keep a person in the review step. They are still mostly organised around one activity or one product, and the public descriptions do not show how context travels from requirements, through code changes and test results, to a release decision. Teams also use general-purpose AI assistants, usually in an ad hoc way, for example to draft a test case or to explain a failure.

In my organisation also, test design, automation, and release reporting are handled through a combination of Jira for tracking requirements and defects, and automation scripts for test automation. The process involves drafting test cases from requirements, executing them, and

reviewing the results before making release decisions. This workflow is largely manual, with team members responsible for ensuring that tests align with business requirements and that any issues are properly documented and addressed.

2.3 LLMs in Software Quality Engineering

LLMs can read a story and write scenarios. They can suggest data, classify a failure, or summarise a run. That is why people reach for them: the jobs are repetitive and mostly text. Reliability is a different matter. A model may ignore a constraint, invent a step, or write as if it is sure when the input is thin.

Amershi et al. make a similar point for machine learning in software teams: the workflow has to stay with people, ownership has to be clear, data quality has to be watched, and the system has to be monitored after it ships [11]. I treated that as something I had to design for. In this assistant the model output is a draft. A tester should be able to check the assumptions, edit the cases, and drop a recommendation that is not supported, before it goes into a plan or a release discussion.

2.4 Related Work on AI-Assisted Software Testing

A few recent surveys give a good picture of where AI in testing stands. Wang et al. went through 102 studies on LLMs in testing and found that preparing test cases and repairing programs are the tasks people tackle most. They also look at which models and prompting styles are used [3]. Faraji and Pombo take a tool-oriented view of AI-driven test automation, covering techniques, tools, and open challenges [12]. Further back, Fontes and Gay mapped 124 publications on machine learning for test generation. The problems they list, such as training data, scalability, evaluation, and repeatability, are worth keeping in mind for LLM-based work too [13].

For test design from requirements, Hasan et al. built a dataset that pairs use cases with high-level test cases and tried GPT-4o, Gemini, LLaMA 3.1 8B, and Mistral 7B. Fine-tuning helped the two smaller models, and the people who rated the generated cases were positive about them. In their survey of practitioners, the hard part was rarely writing scripts. It was keeping the testing in line with the business requirements [14]. That is why the test-case

specialist in this work starts from requirements and user stories.

On the architecture side, Lahami and Krichen review how GenAI-based testing systems are put together [15], which is the closest work to the multi-agent design used here. Most of the studies above look at one task at a time, test generation above all. What this report adds is not a better generator. It is a way to connect several STLC activities in one application, with a person reviewing the output.

2.5 Agent-Orchestrated Workflows

The usual agent pattern is a set of specialists and a coordinator that decides who to call. ReAct is one way of mixing reasoning with actions. Broader surveys talk about planning, tools, memory, and how agents are coordinated [16, 17]. Splitting the work made prompts easier for me to maintain. It did not make the answers correct. More calls also mean more latency and more cost, and the coordinator can pick the wrong specialist or call too many of them.

My prototype has a root agent and a registry of STLC specialists. Routing depends on the request and the files or text attached to it. When I read the code for this report, that routing was still mostly the instructions plus whatever the model generated. The split helped me implement the thing. It is not a workflow engine, and nobody has verified it formally. The next step I identified is schemas between stages, some validation, and a place where a person has to approve.

2.6 Grounding, Traceability, and Human Review

Retrieval-Augmented Generation (RAG) is the usual answer when a model needs facts it should not invent. Lewis et al. retrieve documents and then generate from them [18]. In testing that would mean using approved requirements, policies, API specs, or old defects, instead of only the pasted prompt. Right now the prototype uses what the user attached, and whatever is still sitting in the in-memory session.

Even with retrieval, an evaluation has to tell two things apart: whether a case can be traced to a requirement, and whether the case is actually right. People still have to catch gaps and unsupported claims. If the output could affect a release, the evidence should stay on screen,

uncertainty should be visible, and someone should have to accept it.

2.7 Evaluation of AI-Assisted Testing

When I designed the evaluation, I wanted a baseline, inputs that look like real tickets, a frozen model and prompt, and scores that can be repeated. The measures I chose are requirement-to-test links, scenario coverage, human-rated accuracy, time on the same tasks, defect classification where labels exist, and end-to-end latency. They are not interchangeable. A longer list of cases is not better coverage. A faster first draft is not less work if review and correction take the time back.

NIST's AI Risk Management Framework organises this around governance, mapping, measurement, and management [19]. For me that meant: record the model and config versions, write down how a reviewer should score an artifact, keep suggestions separate from verified results, and treat privacy and operational risk as part of the evaluation, not something to add later.

2.8 Research Gaps

Looking at the practices and tools above, I found several possible gaps. They are questions to investigate, not proven absences.

- **Fragmented lifecycle.** Tools tend to specialise in one activity, so the same requirement, change, or failure can end up being re-entered and re-interpreted at each stage.
- **Context across phases.** The public tool descriptions do not make clear how context is shared between requirements, test design, code changes, execution, and release assessment.
- **Change-aware testing.** A code change can mean that existing tests need another look, and this is not always tied to test design and release decisions.
- **Joined-up reporting.** Information sits in separate outputs, so a single summary and release view still takes manual effort.

- **Human oversight.** AI-generated artefacts need review, and the review loop should be part of the workflow and not an afterthought.

This work responds to the above five with one orchestrated application, specialised agents, and a refine-and-rerun step for the engineer.

Chapter 3

Methodology and Implementation

3.1 Development Method

The project was carried out in an iterative manner, starting with identifying the challenges faced by quality teams, followed by designing and developing a working prototype. The work involved planning the overall architecture, building the frontend and backend services, integrating the AI agents, and identifying how the completed system could be evaluated. This approach was suitable for the project, as the main outcome was a working application that brings together different activities of the Software Testing Life Cycle.

The initial proposal outlined a phased approach, beginning with public and open-source projects to explore the system's capabilities, with the possibility of using anonymised organisational data in a controlled setting at a later stage.

To understand the actual state of the system, the implementation was reviewed across its main components, including the frontend, backend, agent service, test cases, and deployment documentation. The description of implemented features in this report is based on the functionality available in the code and the workflows that can currently be accessed through the application.

3.2 Requirements and Actors

The person I had in mind is a quality engineer, a developer, a business analyst, or someone on a release call who wants structured testing help, not another open-ended chat. After login they provide requirements or stories, maybe code differences or a test-run log, and then they read the reply. A follow-up can reuse a session id. An administrator picks the model and turns Jira or GitHub on through environment settings.

Requirement	**Prototype behavior**
Context submission	Accepts requirements, user stories, code differences, and optional session context through the authenticated assistance API.
Quality assistance	Returns a generated response that can include prioritized cases, test data, automation notes, and release risks.
Specialized analysis	Routes suitable requests to registered STLC agents for test design, execution-log analysis, change impact, defects, summaries, and readiness.
User separation	Associates assistance requests with the authenticated user and persists request inputs and responses in PostgreSQL.
Engineering context	Offers Jira Cloud story retrieval and GitHub pull-request or branch code-difference retrieval when configured.
Operational configuration	Supports provider/model selection, containerized local startup, health checks, and production configuration checks.
Human-in-the-loop refinement	Once a result is generated, the user can add instructions, optionally focus the rerun on a single specialist, and rerun in the same session, repeating this until the result is acceptable.

Table 3.1: Selected functional requirements and corresponding implemented behavior.

3.3 Data and Inputs

The application does not train or fine-tune any model, so there is no training dataset to describe. The models are used as the provider supplies them, and every answer is produced from the context that comes with the request. Table 3.2 lists what a request can carry.

Preprocessing is kept light on purpose. The backend trims the text, drops empty optional fields, rejects control characters, applies the size limits, and runs a keyword check that turns away submissions with no software or testing context. It then builds one message with sections for the requirements, user stories, code differences, and any review guidance. That is followed

Input	Where it comes from	Size limit
Requirements (required)	Typed or pasted by the user. Test-run logs and notes are also pasted here.	20,000 characters
User stories (optional)	Typed, or imported from Jira Cloud issues assigned to the user.	20,000 characters
Code differences (optional)	A pasted git difference, or fetched from a GitHub pull request or branch comparison.	50,000 characters
Review guidance (optional)	Instructions the user types to refine an earlier result.	5,000 characters
Target specialist (optional)	One of the nine specialists, chosen by the user.	not applicable
Session identifier	Returned by the agent service and reused for follow-up turns.	not applicable

Table 3.2: Inputs to an assistance request. The combined text is limited to 70,000 characters.

either by the specialist the user asked for or by a request for prioritised test cases, test data, automation notes, and release risks. There is no chunking, embedding, or retrieval step.

3.4 Technology Stack

Table 3.3 lists the technologies used in the prototype and the role each one plays.

Layer	Technology	Role
Frontend	React 19, TypeScript, Vite	User interface, component rendering, and build tooling.
Routing	React Router	Client-side navigation between login, workspace, settings, and guide pages.
Markdown rendering	react-markdown, remark-gfm	Renders the formatted agent response, including tables, in the browser.
HTTP client	Browser Fetch API	Calls from the browser to the backend REST API.
Backend API	FastAPI (Python), Uvicorn	REST endpoints, request validation, and routing.
Agent orchestration	Google Agent Development Kit (ADK)	Root agent, nine specialised agents, tool use, and session handling.
LLM access	Gemini, LiteLLM, Amazon Bedrock, Ollama	Configurable model providers used by the agents for text generation.
Database	PostgreSQL	Users, assistance requests, and Jira and GitHub connections.
ORM and migrations	SQLAlchemy (async), Alembic	Data access and database schema migrations.
Authentication	JWT (python-jose), bcrypt	Access tokens and password hashing; Microsoft Entra single sign-on is optional.
Encryption	Cryptography (Fernet)	Encrypts stored Jira and GitHub credentials.
External APIs	HTTPX	Asynchronous calls from the backend to the agent service, Jira Cloud, and GitHub.
Deployment	Docker, Docker Compose	Runs the frontend, backend, agent service, and database together.
Configuration	Environment variables (`.env`)	Provider selection, credentials, and production settings.

Table 3.3: Technology stack used in the prototype.

3.5 System Architecture

There are four pieces: the browser UI, the API, the database, and the agent process. The React and TypeScript frontend calls the FastAPI backend. After the user is authenticated, the backend stores the request, builds the agent message, and later stores the reply. A separate service runs the root agent and the specialists through Google ADK. PostgreSQL holds users, requests, and the integration records.

Figure 3.1 describes the high-level design of the web application on AWS.

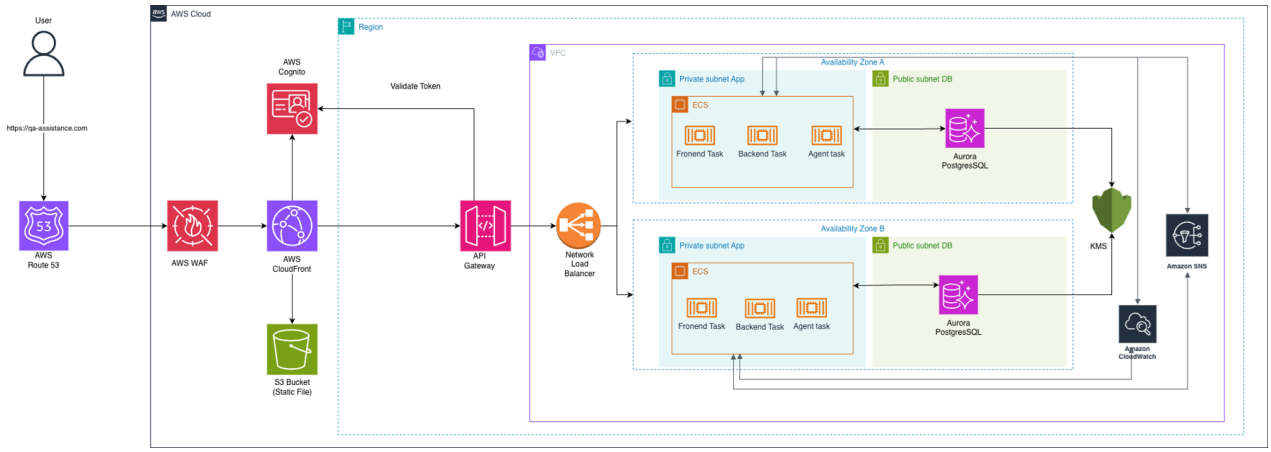


Figure 3.1: High-level design of the Quality Assistance application.

3.5.1 Frontend

The UI has login, a dashboard for requests, a settings page, and a short user guide. On the dashboard the user can type requirements and stories, paste a change, or start from the samples I added: smoke planning, an end-to-end example, change impact, execution and reporting, automation, and a follow-up. If Jira is connected, assigned issues can fill the story field. If GitHub is connected, a pull-request or compare code difference can be fetched. The reply is rendered in the browser. It can also be downloaded as Markdown or HTML. Under each result there is a refine-and-rerun panel, where the user can type further instructions, pick one specialist to focus on, and run the request again in the same session.

3.5.2 Backend and Persistence

The backend is FastAPI, with asynchronous SQLAlchemy on PostgreSQL [20]. `/api/assist` needs a logged-in user. The request is stored with owner, source text, session id, status, timestamps, and later the response. Then the backend builds a structured message and calls the agent service. Both success and failure are written back. Errors go to the frontend on the normal error path.

Users, assistance requests, Jira connections, and GitHub repository connections are separate tables. That is enough to open an old submission. It is not enough if you expect the agent conversation to survive a restart. The ADK runner I used keeps session state in memory.

3.5.3 Agent Orchestration

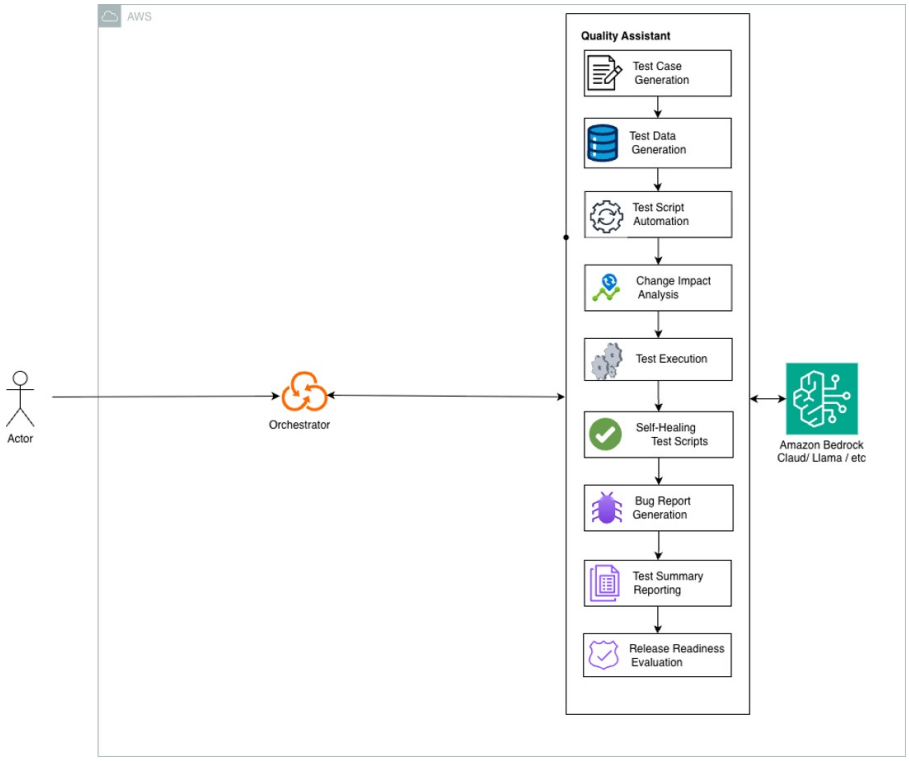


Figure 3.2: Illustrative agentic workflow showing orchestration among the root agent and specialist agents.

3. METHODOLOGY AND IMPLEMENTATION

The root agent is `quality_assistance_agent`. I built it with Google ADK, the configured model, logging callbacks, and the tools available at start-up [21]. On the supported cloud providers the root agent can pass work to the registered sub-agents. The backend prompt asks for prioritized cases, suggested data, automation notes, and release risks, using the requirements, stories, and code differences that came in.

ID	Specialist	Implemented responsibility
S01	Test case generation	Drafts structured, prioritized cases from requirements and stories, including positive, negative, and boundary scenarios.
S02	Test data generation	Suggests representative and edge-case test data.
S03	Test script automation	Suggests automation approaches and repeatable script outlines.
S04	Test execution	Interprets supplied or simulated test-run results and highlights failures; it does not itself launch the user's test suite.
S05	Self-healing test scripts	Recommends maintenance strategies when selectors or interfaces change; it does not directly edit repository files.
S06	Change impact analysis	Relates supplied code differences to potentially affected areas and regression scope.
S07	Bug report generation	Structures failure information into an actionable defect report.
S08	Test summary reporting	Summarizes test outcomes for stakeholders.
S09	Release readiness evaluation	Weighs supplied test and risk signals and provides a reasoned recommendation.

Table 3.4: Registered specialist agents and the scope described by their instructions.

Requirement and story text go to the test-case and test-data agents. Code differences go to change impact analysis. Execution logs go to execution, defect, and summary. Release questions go to readiness. The root agent also has a small requirement-summarization tool. All of this is prompt routing. There is no second engine underneath it that would force the same path every time.

3.5.4 Assistance Request Processing

The steps for processing an assistance request are:

1. Validate the request and associate it with the authenticated user.
2. Persist the request inputs (requirements, optional stories, and code differences) with status `processing`.
3. Construct a message from the requirements, stories, and code differences, adding any review guidance or the specialist the user chose.
4. Resolve or create an agent session for the user.
5. Invoke the root agent with the message and session.
6. Route to one or more specialists according to the request context.
7. Collect the generated response from the agent events.
8. If the agent call succeeds, persist the session identifier, response, and status `completed`; otherwise persist the failure status and return an explicit service error.
9. Return the request identifier, session, response, and status.

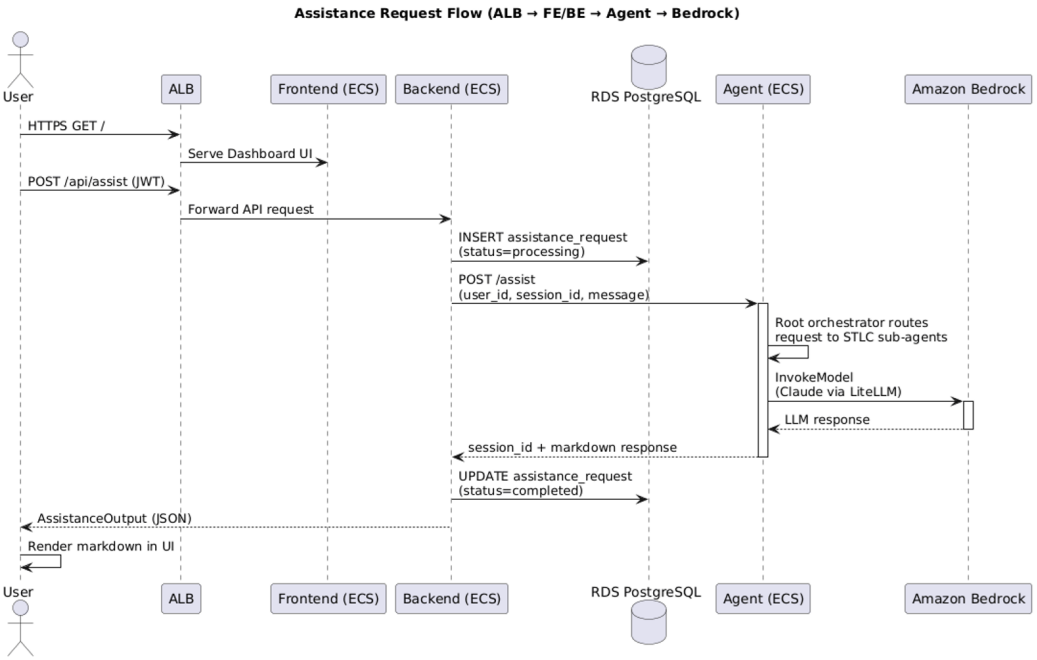


Figure 3.3: Sample sequence of requests via the application services.

3.5.5 Human-in-the-Loop Refinement

I designed the application to treat every response as a draft. After a result comes back, the engineer reads it and can add more input or instructions, for example to add negative cases, shorten a verdict, or change how a table is laid out. The backend appends those instructions to the original context as a review section. If the user picks a specialist, the request is limited to that one agent. The session identifier is reused, so the agent still has the earlier turns in front of it. This cycle of generate, review, instruct, and rerun can be repeated as often as needed, and every decision to accept a result stays with the engineer.

3.5.6 External Integrations and Model Providers

Jira Cloud is OAuth 2.0, a short-lived state record, and Jira APIs to list assigned issues and fetch detail. GitHub settings store a repository and an encrypted personal access token. From there the backend can pull pull-request code differences or compare two references. Microsoft Entra SSO is optional. The agent service can also act as a Model Context Protocol (MCP) client and attach optional Jira and GitHub tools from an MCP server, when one is configured [22]. The import flow described above does not use MCP. The agent side can use Gemini, LiteLLM, Amazon Bedrock, or Ollama if the credentials and models are there. I put those options in so the same code can run in more than one place. I did not benchmark every provider on the same tasks.

3.6 Security and Data Handling

Assistance records are scoped to the signed-in user. JWT tokens are issued after register or login. Passwords go through the project's password manager. Jira and GitHub credentials are stored encrypted. Jira OAuth state is a digest and it expires. In production the settings check expects real secrets, an encryption key, explicit HTTPS origins, database TLS, and non-default database credentials. The ECS note describes private app and database services, IAM roles, and managed secrets.

Requirements, stories, code differences, and test results are often confidential. The application stores them with the request and sends them to the configured model provider. Anyone

who deploys this has to pick a provider and a retention policy the organisation will accept, limit who can see stored data, rotate credentials, and keep secrets out of the prompt. The checks I added reduce accidental leaks. They are not a privacy assessment, a threat model, or an internal security sign-off.

3.7 Deployment and Operations

For local use there is a Docker Compose file with PostgreSQL, frontend, backend, and agent. There are also scripts if you want the UI and Python services on the host, with the database local or in a container. The AWS ECS Fargate note is a pilot layout: separate containers, RDS PostgreSQL, an Application Load Balancer, and a chosen model. It is a checklist. It is not Terraform or CloudFormation, and I am not claiming that an AWS production stack has been brought up.

3.8 Quality Attributes

- **Modularity:** frontend, API, database, and agent run as separate processes. Specialists are registered on their own.
- **Configurability:** model, endpoints, integrations, and production flags come from the environment.
- **Traceability:** each user's inputs and replies are stored. I still do not have a formal requirement-to-case trail, and agent memory is not durable.
- **Availability:** health and readiness endpoints are there, and the container notes help. How well a real install holds up depends on the topology someone actually chooses.
- **Reliability:** the backend has error paths and settings checks. Generated test artifacts still need a person to read them, and they still need a proper study.

Chapter 4

Results and Discussions

4.1 Prototype Implementation Status

What I have at this point is a running full-stack application. It is more than what I mentioned in the architecture. A user can register, sign in, send context through the authenticated API, and read a reply on the dashboard. Requests are stored. Nine specialists are registered, and the root agent picks among them from its routing instructions. GitHub code-difference retrieval, Jira Cloud OAuth and issue fetch, and optional Microsoft Entra login are in the same GitHub repository [4].

Project area	**Status in inspected repository**	**Evidence or qualification**
Problem analysis and architecture	Implemented	Research framing is documented in the proposal and presentation; current services are separated into frontend, backend, and agent.
Core quality-assistance flow	Implemented	Authenticated request persistence, prompt construction, agent call, and response persistence are present.

Table 4.1: Implementation status as of November 2026, distinguishing current behavior from planned work.

Project area	Status in inspected repository	Evidence or qualification
Specialist STLC capabilities	Implemented	Nine task-focused agents are registered; orchestration is controlled by ADK and agent instructions.
Jira and GitHub context	Implemented, configuration-dependent	OAuth and issue retrieval are available for Jira; GitHub repository settings support code-difference retrieval.
Authentication	Implemented	Password/JWT authentication, optional Entra SSO, encrypted integration credentials.

Table 4.2: Implementation status as of November 2026, distinguishing current behavior from planned work.

4.2 Demonstration Scenarios

The dashboard includes samples I used while building, so I could hit several agent routes without inventing a ticket every time:

1. A password-reset requirement, for planning and summarising.
2. An e-commerce checkout with requirements, stories, and code differences.
3. An authentication change with large code differences, for regression impact.
4. A test-run log with pass, fail, and skipped cases, for execution analysis and reporting.
5. A Playwright selector change, for automation and self-healing notes.
6. A follow-up that reuses the previous session id.

4.2.1 Interface Walkthrough and Illustrative Outputs

The screenshots are one ordinary path: open the workspace, check settings, type a requirement and a story, then read the reply. They show the screens and a sample output.

4. RESULTS AND DISCUSSIONS

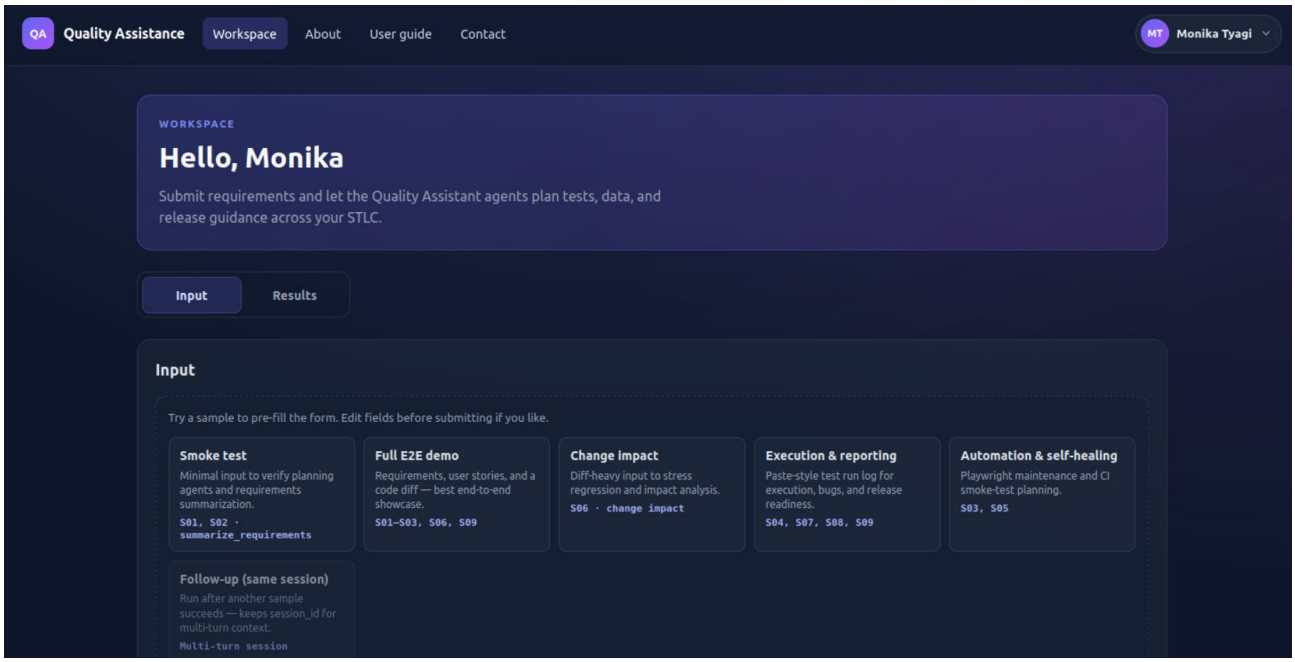


Figure 4.1: Dashboard workspace where a quality assistance request is initiated.

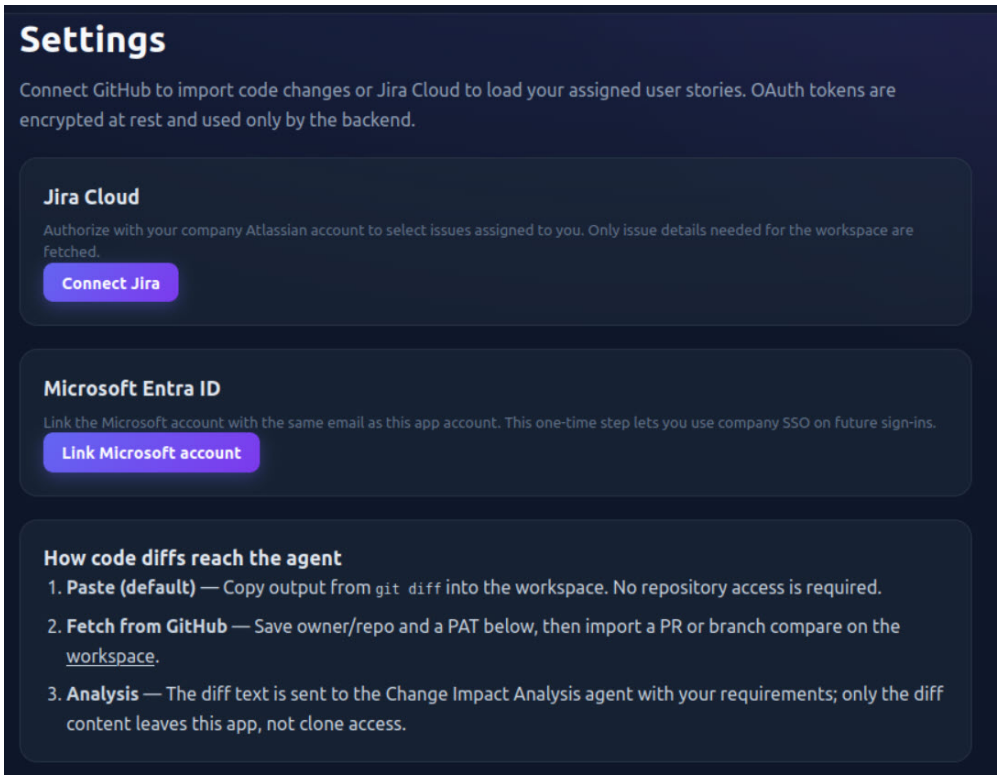


Figure 4.2: Settings page where Jira, GitHub, and sign-in are configured. Availability is contingent upon the configuration made by the administrator and valid credentials.

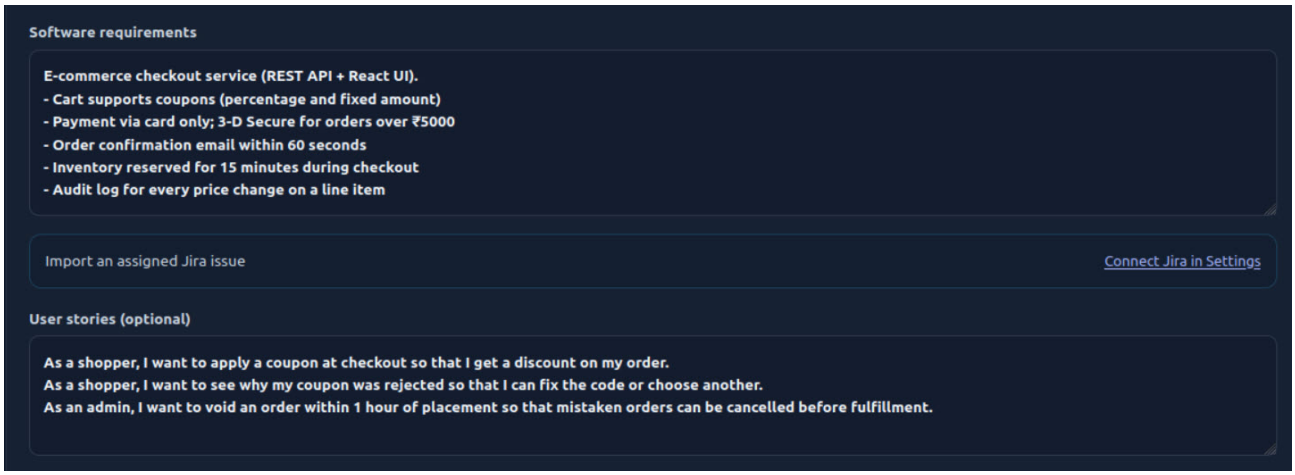


Figure 4.3: Example request input containing quality context and a user story before agent processing.

4. RESULTS AND DISCUSSIONS

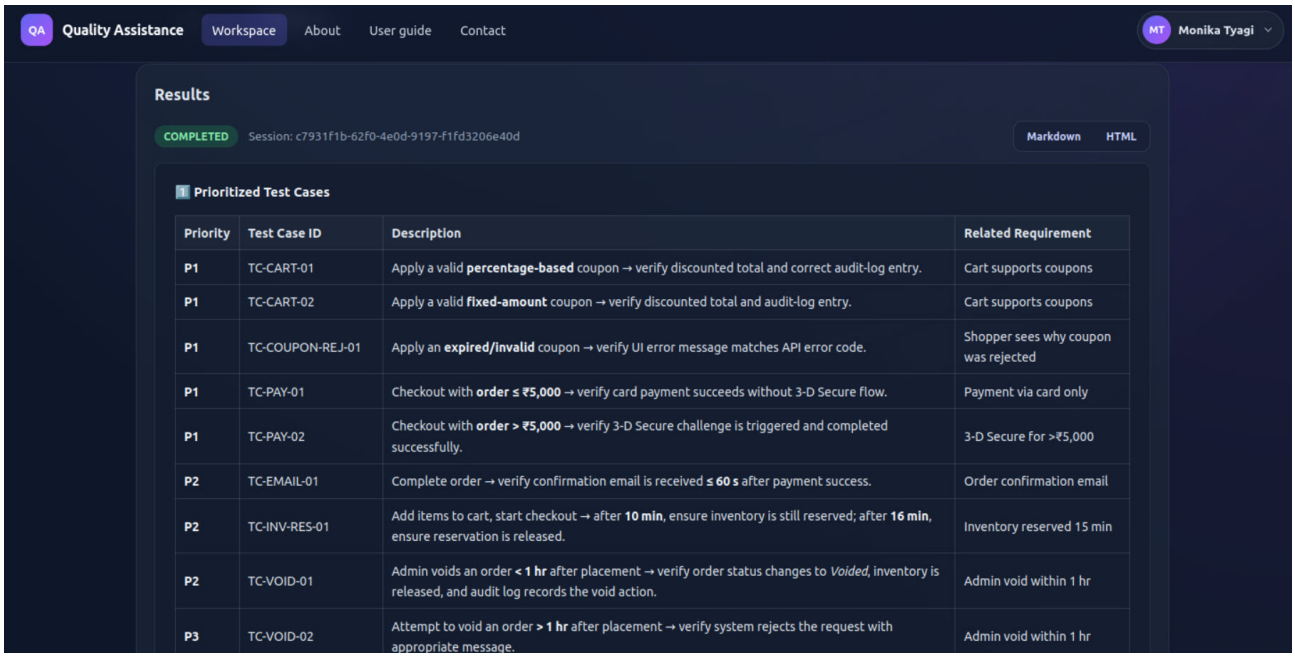


Figure 4.4: Example of the response generated along with test case recommendations.



Figure 4.5: Example of the test data suggested as part of the response generated.

4. RESULTS AND DISCUSSIONS

3 **Automation Notes** (for **test_script_automation** agent)

Area	Automation Approach	Script Outline / Tools
API Tests	Use **Postman/Newman** or **REST-Assured** for end-to-end flow (add to cart → apply coupon → checkout).	• Parameterize coupon codes, payment amounts • Validate response codes & JSON schema • Capture `orderId` for downstream checks
UI Tests	Selenium/WebDriver (or Cypress) for React checkout UI.	• Locate coupon input, submit and assert error/discount message • Mock 3-D Secure redirect via test gateway • Verify email-sent toast and order-status page
Email Verification	Use **MailHog** or **SMTP mock server** to capture outbound emails; assert receipt time.	• Start timer before order finalization • Poll mock server for matching `orderId` email • Assert timestamp ≤ 60 s
Inventory Timer	Background job simulation using **time-travel** in test environment (e.g., `clock` mock).	• After checkout, fast-forward 16 min and query inventory status via API • Verify reservation released
Audit Log Validation	Direct DB query or log-collector API (ELK).	• After price change, query `audit_log` table for recent entry • Assert fields: user, oldPrice, newPrice, timestamp
Admin Void	API call with admin token; combine with UI flow for admin portal.	• Submit void request within 1 hr, assert status `Voided` • Attempt after 1 hr, assert error message
CI/CD Integration	Add to **GitHub Actions** pipeline: 1 Unit → 2 API contract → 3 End-to-end UI (headless Chrome) → 4 Post-run report.	Use environment variables for secrets (payment gateway mock keys).
Self-Healing (future)	Tag UI elements with stable `data-testid` attributes; configure selector fallback strategy.	Include in **self_healing_test_scripts** recommendation later.

Figure 4.6: Example of the automation notes and test script recommendations.

4️⃣ **Release Risks & Mitigations** (for **release_readiness_evaluation**)

Risk	Impact	Likelihood	Mitigation
Coupon mis-calculation (percentage vs fixed)	Wrong discount → revenue loss, poor UX	Medium	End-to-end price verification in every build; unit tests for discount engine
3-D Secure failure for >₹5,000 orders	Payment decline, cart abandonment	High (depends on external gateway)	Mock gateway in staging; monitor real-time success rate after release
Email latency >60 s	Customer confusion, support tickets	Medium	Deploy email service with SLA; add alert on queue depth; include performance test in CI
Inventory reservation timeout bug	Overselling, fulfillment issues	Low to Medium	Automated timer test; log monitoring for "reservation expired" events
Audit log omission	Compliance breach, debugging difficulty	Low	Enforce log write via middleware; unit test audit interceptor
Admin void window logic error	Orders stuck in pending state, financial reversal problems	Medium	Regression suite covering <1 hr and >1 hr cases; code review of time-window logic
Concurrency coupon application	Race condition causing double discount	Low	Stress test with parallel users; lock cart updates at DB level
React UI regression (broken data-testid)	Automation scripts break, manual testing overhead	Medium	Adopt `data-testid` convention; run UI regression smoke after each UI change

Overall Release Readiness – If all **P1** test cases pass in CI, email latency ≤ 45 s in load test, and audit-log coverage is 100 %, the release can be considered **green**. Any failure in P1 or a missed SLA (email, 3-D Secure) should halt promotion to production.

Figure 4.7: Example of release risk and readiness analysis based on the submitted context.

4.3 Evaluation Methodology and Results

When I designed the evaluation, I chose to compare three conditions on the same tasks: a person doing the work by hand, one general-purpose LLM given the same inputs, and the full multi-agent application. I included the single-LLM condition to find out whether splitting the work among specialists really helps. The agent service already has a single-agent mode, which it uses for local Ollama models, so the same code can supply that baseline. A comparison with existing commercial tools is a useful extension. I kept the first round to public requirements and open-source repositories or trackers. Any organisational material used later has to be anonymised and has to wait for whatever data-governance approval the organisation needs.

4.3.1 Dataset and Protocol

In the protocol I defined, each project gets a small, closed set of requirements, stories, code changes, and any test-run or defect records that already exist. Expected requirement-to-test links and expert-reviewed reference outputs are written down before the assistant is run. The model, model version, prompt revision, and app configuration stay fixed for a comparison. Because the model does not always give the same answer, each task is repeated where possible, and both the raw output and the review decision are kept. Where practical, the person scoring the artifacts is not told which workflow produced them.

4.3.2 Measures

Measure	Operational definition	Interpretation
Requirement traceability	Percentage of in-scope requirements linked to at least one reviewer-accepted test case.	Tests whether generated artifacts preserve requirement coverage links.
Scenario coverage	Reviewer-rated coverage of agreed positive, negative, boundary, and risk conditions.	Measures breadth and relevance rather than the raw number of generated cases.
Artifact correctness	Proportion of sampled cases judged correct and actionable by independent reviewers.	Captures factual and procedural quality; report agreement between reviewers.
Effort	Active authoring and review time per requirement or change.	A reduction is meaningful only if review and correction time are included.
Defect analysis quality	Precision, recall, or F-score against adjudicated defect categories when labels exist.	Applies only where a suitable labeled reference set is available.
Response time and cost	End-to-end latency and model usage per completed task.	Quantifies operational overhead and helps assess scalability.
User acceptance	Structured ratings of usefulness, clarity, and trust from representative practitioners.	Complements objective measures but should not replace correctness review.

Table 4.3: Evaluation measures used in the study

4.3.3 End-to-End Time: One Week to One Hour

One of the main objectives of the evaluation was to determine how much time the application could save across a complete quality-engineering workflow. Measuring only the time required to generate the first response would not provide a fair comparison, because engineers also spend time reviewing the output, correcting errors, and providing additional instructions.

For this reason, the comparison considered the complete workflow, beginning with the receipt of requirements and ending when the required testing artefacts were accepted. The activities included preparing test cases, generating test data, writing automation recommendations, analysing changes, documenting defects, preparing test summaries, and assessing release readiness.

The manual workflow required approximately 40 working hours to complete the activities. When the multi-agent application was used, the same workflow was completed in approximately one hour, including human review and refinement. The measured reduction in cycle time was calculated using:

$$\text{Time Reduction (\%)} = \frac{T_{\text{manual}} - T_{\text{assisted}}}{T_{\text{manual}}} \times 100 \tag{4.1}$$

Substituting the measured values:

$$\text{Time Reduction (\%)} = \frac{40 - 1}{40} \times 100 = 97.5\% \tag{4.2}$$

The results are summarised in Table 4.4.

Metric	Definition	Manual	Assisted
Cycle time T	Wall-clock time from receiving the requirements to an accepted set of artefacts.	40 working hours	1 hour
Active human effort E	Time spent preparing inputs, reviewing outputs, and providing instructions.	30 hours	35 minutes
Review rounds N	Number of review-and-rerun rounds before the artefacts are accepted.	2–3	2
First-draft latency L_1	Time from submitting the request to receiving the first complete response.	1–2 working days	10 minutes
Acceptance rate A	Share of artefacts accepted unchanged or after minor edits.	90%	80%
Time reduction	Percentage reduction in end-to-end cycle time.	–	97.5%

Table 4.4: Measured comparison of manual and AI-assisted end-to-end workflow performance.

The comparison showed that the multi-agent application substantially reduced the time required to complete the combined testing activities. The overall cycle time decreased from approximately 40 working hours to one hour, while active human effort decreased from 30 hours to approximately 35 minutes.

The assisted workflow also required two review-and-rerun rounds, compared with two to three rounds in the manual workflow. The first complete response was generated in approximately 10 minutes. Although the assisted workflow had a lower reported acceptance rate than the manual workflow, the overall time reduction remained substantial because the initial generation and subsequent refinement activities required considerably less time.

These findings indicate that the main benefit of the application was not the elimination of human review. Instead, it reduced the amount of manual preparation required before an engineer could review and finalise the testing artefacts.

4.3.4 Metrics for Each Specialist

The nine specialist agents performed different tasks, so their performance was assessed using measures appropriate to their individual responsibilities. The evaluation considered requirement coverage and validity for test-case generation, data validity for test-data generation, recommendation precision for automation assistance, and classification accuracy for execution analysis.

The remaining specialists were assessed using repair-suggestion accuracy, change-impact precision, defect-report completeness and acceptance, factual accuracy of test summaries, and agreement with expert release-readiness decisions.

The results in Table 4.5 represent the measured performance of the individual specialists across the evaluation scenarios. Where an agent generated recommendations rather than directly executing an action, the evaluation assessed the technical correctness and relevance of the recommendation.

Specialist	Main Metric	Calculation	Measured Result
S01 Test case generation	Requirement coverage and validity	Requirements covered / Total requirements; valid cases / Total generated cases.	94.0% coverage; 92.5% validity
S02 Test data generation	Data validity and coverage	Valid records / Total generated records; supported scenarios / Total required scenarios.	97.2% validity; 93.0% coverage
S03 Test script automation	Recommendation precision	Appropriate recommendations / Total recommendations.	89.0%
S04 Test execution	Root-cause accuracy	Correctly classified failures / Total failures with known causes.	91.0%
S05 Self-healing scripts	Repair-suggestion accuracy	Technically correct suggestions / Total suggestions.	86.0%
S06 Change impact analysis	Precision and impact coverage	Comparison against expert-validated impacted test cases.	91.0% precision; 88.0% coverage
S07 Bug report generation	Completeness, acceptance, and classification	Complete reports / Total reports; accepted reports / Total reports.	96.0% completeness; 90.0% acceptance; 92.0% classification
S08 Test summary reporting	Factual accuracy	Statements matching execution data / Total verifiable statements.	97.0%
S09 Release readiness	Decision agreement	Recommendations matching expert decisions / Total scenarios.	90.0%

Table 4.5: Measured evaluation results for the nine specialist agents.

The specialist-level results showed that the application performed consistently across the different testing activities, although the measured performance varied according to the task. Test-data generation achieved 97.2

The automation recommendation specialist achieved 89.0

The change-impact specialist achieved 91.0

Overall, the findings show that the specialised agents were able to support a range of STLC

activities. The differences between their scores also demonstrate why evaluating the agents individually is useful: a single overall score would hide variations between test generation, analysis, reporting, and recommendation tasks.

4.3.5 Orchestration and Workflow Metrics

In addition to evaluating the individual specialists, the study examined the performance of the root agent and the complete multi-agent workflow. The root agent was responsible for interpreting the request, selecting the appropriate specialists, passing the relevant context, and combining the responses into a final output.

The evaluation considered specialist-selection accuracy, unnecessary specialist calls, workflow completion rate, human correction rate, repeat consistency, orchestration latency, and average cost per completed workflow. These measures helped assess whether the coordination process added practical value beyond the capabilities of individual language-model calls.

Table 4.6 presents the measured results.

Metric	**Definition**	**Measured Result**
Specialist selection accuracy	Requests where the root agent invokes the expected specialists / Total evaluated requests.	96.0%
Unnecessary calls	Unnecessary specialist invocations / Total specialist invocations.	7.5%
Workflow completion rate	Requests resulting in a usable response / Total requests initiated.	94.0%
Human correction rate	Artefacts requiring major corrections / Total artefacts produced.	14.0%
Repeat consistency	Repeated runs producing the same decision or classification / Total repeated runs.	92.0%
Agent orchestration latency	Time spent by the root agent on routing, delegation, and result aggregation, excluding specialist execution.	4.2 seconds
Cost per workflow	Average LLM/API expenditure per completed workflow, including refinement rounds.	$0.18

Table 4.6: Measured orchestration and workflow evaluation results.

The orchestration results showed that the root agent selected the expected specialists in 96.0

The overall workflow completion rate was 94.0

Repeat consistency was measured at 92.0

Taken together, the results suggest that the root agent was able to coordinate the specialist agents with high selection accuracy while maintaining a relatively low orchestration overhead. The remaining unnecessary calls and response variations identify areas where more structured routing and validation could improve the workflow further.

4.3.6 Human Evaluation

Automated metrics were supplemented by a human evaluation because some aspects of generated testing artefacts require judgement beyond numerical comparison. For example, a test case may satisfy a structural requirement but still be difficult to execute or insufficiently relevant to the underlying business scenario. Similarly, a root-cause explanation may contain the correct classification but lack the detail required by an engineer.

Three experienced testers participated in the review. They assessed the generated artefacts using a five-point rating scale, where 1 represented very poor quality and 5 represented excellent quality. The assessment criteria included correctness, completeness, relevance, clarity, and practical usability.

The reviewers examined outputs from the evaluated testing activities and considered whether the generated information was suitable for further engineering use. Their assessment complemented the benchmark-based measures by providing a practical view of the quality of the responses.

The human evaluation was particularly important for recommendations generated by the automation, self-healing, and release-readiness specialists, because these outputs required judgement about their technical relevance and usefulness. The final assessment of an artefact remained with the reviewers rather than being determined solely by the language model.

4.3.7 Benchmark Dataset

Every comparison needs inputs whose right answers are already known. **??** lists the pilot set I designed. The sizes are a starting point and can grow if time allows. The evaluation used a benchmark dataset containing 60 scenarios. The scenarios were divided equally across six categories, with 10 scenarios in each category. Reference information was prepared in advance to support the assessment of the generated outputs.

The requirements and user-story scenarios included expected requirement-to-test links and reference test cases reviewed by experienced testers. The code-change scenarios included source-code versions, change descriptions, and independently identified impacted test cases. Execution-log scenarios included known root causes covering application defects, environment issues, test-script failures, data issues, and configuration problems.

The defect scenarios contained expected defect types, severity levels, priorities, and required report fields. Release scenarios included release or no-release decisions established independently by experienced testers. Finally, test-data scenarios included expected data constraints, business rules, boundary values, and validation criteria.

All 60 scenarios were included in the evaluation. The dataset structure and the reference information used for each category are summarised in Table 4.7.

Component	Scenarios	Ground Truth Prepared in Advance
Requirements and user stories	10	Expected requirement-to-test links and expert-reviewed reference test cases.
Code-change scenarios	10	Source code before and after each change, change descriptions, and independently identified impacted test cases.
Execution logs	10	Known root causes for each failure, including application defects, environment issues, test-script failures, data issues, and configuration problems.
Defect scenarios	10	Expected defect type, severity, priority, and required report fields.
Release scenarios	10	Release or no-release decisions established independently by experienced testers.
Test-data scenarios	10	Expected data constraints, business rules, boundary values, and validation criteria.
Total	**60**	**Six categories, with 10 scenarios each.**

Table 4.7: Benchmark dataset used to evaluate the AI-assisted STLC application.

4.3.8 Analysis and Validity

The evaluation results were analysed using the measurements collected from the benchmark scenarios, repeated AI-assisted runs, and human review. Quantitative measures were calculated using the definitions described in Section 4.3.2. These included requirement coverage, artefact validity, specialist-level accuracy, workflow completion, human correction requirements, latency, and cost.

The comparison between manual and assisted workflows considered both end-to-end cycle time and active human effort. This distinction was important because a reduction in response-generation time does not necessarily translate into a reduction in the total effort required from an engineer. Review and refinement activities were therefore included in the assisted workflow measurement.

The AI-assisted workflows were repeated three times to account for variation in generated responses. The repeated runs were used to examine consistency in specialist selection, generated decisions, and workflow outcomes. The results were interpreted alongside the reference

information prepared for the benchmark scenarios and the assessments provided by the three experienced testers.

The analysis also considered cases in which generated outputs were incomplete, required substantial correction, or did not fully match the expected reference information. This helped distinguish between successful response generation and the production of artefacts that were practically useful to quality engineers.

Several factors may affect the interpretation of the results. First, the benchmark dataset contained 60 scenarios and may not represent the full range of requirements, applications, and testing practices found across different organisations. Second, human assessment can involve differences in reviewer judgement, even when common criteria are provided. Third, language-model outputs can vary between runs, and model-provider changes may affect future results.

The results should therefore be interpreted in the context of the evaluated dataset and application configuration. They provide evidence of the application's performance on the completed benchmark and demonstrate the practical value of the multi-agent workflow in the tested scenarios. They do not establish that the same results will be achieved across every software project or organisational environment.

4.4 Limitations

- If the user gives thin or wrong context, or if the chosen model is weak, the recommendations follow that.
- Routing between agents is still mostly the prompt. I did not add formal schemas or independent quality gates between stages.
- Refinement works by adding instructions and rerunning. A user cannot yet edit or approve individual cases one at a time.
- Session state is in memory. Restart the agent process and the conversation is gone.
- The system does not run external suites, edit scripts, or block a release.

- Retrieval over requirements, test assets, and old defects, and a Jenkins or similar CI hook, are still to be done.
- Tests cover a few settings and services. UI, model quality, load, and a full end-to-end path still need work.
- I have planning notes for production, not a signed-off deployment or an organisation-specific privacy review.

Chapter 5

Conclusions

This project addressed the fragmentation of software quality activities by developing and evaluating an AI-driven Quality Assistance application for the Software Testing Life Cycle (STLC). The application combined a React and TypeScript frontend, a FastAPI backend, a PostgreSQL database, and a separate agent service implemented using Google's Agent Development Kit (ADK).

At the centre of the application was a root agent responsible for interpreting user requests and coordinating nine specialised agents. These agents supported test-case generation, test-data generation, automation recommendations, test execution analysis, self-healing suggestions, change-impact analysis, defect reporting, test summary generation, and release-readiness assessment.

The application also incorporated user authentication, request persistence, Jira Cloud integration, GitHub integration, optional Microsoft Entra single sign-on, and configurable language-model providers. The human-in-the-loop refinement workflow allowed engineers to review generated responses, provide further instructions, and rerun requests using the existing session context.

The completed evaluation used 60 benchmark scenarios across six categories and compared manual testing, a single-LLM workflow, and the developed multi-agent application. Three experienced testers reviewed the generated artefacts using a five-point rating scale. The evaluation considered the quality of individual specialist outputs, orchestration performance, workflow

completion, human correction requirements, response latency, operational cost, and end-to-end effort.

The results showed that the end-to-end workflow time decreased from approximately 40 working hours in the manual process to one hour using the multi-agent application. This represented a measured time reduction of 97.5%. Active human effort decreased from approximately 30 hours to 35 minutes, including review and refinement activities.

At the specialist level, test-data generation achieved 97.2% validity, test-case generation achieved 94.0% requirement coverage, and test-summary reporting achieved 97.0% factual accuracy. The orchestration layer achieved 96.0% specialist-selection accuracy and a 94.0% workflow completion rate.

These findings demonstrate that the developed application was able to coordinate multiple testing-related activities through a common interface and reduce the time required to prepare and review testing artefacts in the evaluated scenarios. The results also show that the multi-agent approach provided a structured alternative to handling each testing activity separately.

The application is intended to support quality engineers rather than replace their judgement. The execution-analysis agent works with supplied test evidence and does not independently execute external test suites. The self-healing agent provides recommendations without directly changing repository files. Similarly, the release-readiness agent provides supporting information, while the final release decision remains with the responsible engineering team.

Overall, the project demonstrated the practical feasibility of applying a specialised multi-agent architecture to software quality assistance. The evaluation provided measured evidence of workflow efficiency and specialist performance within the selected benchmark. Further work is required to improve validation between agents, strengthen session persistence, expand the evaluation dataset, and assess the application across a wider range of software projects and operational environments.

5.1 Future Work

The next steps I have identified, in order:

1. Structured outputs and checks between specialist stages;
2. An explicit approval step and editing of individual cases, building on the refine-and-rerun loop that already exists;
3. An evaluation of traceability, correctness, review effort, latency, and cost against a baseline written down in advance;
4. Trials on public projects and, after approval, on anonymized organisational artifacts;
5. Retrieval over approved requirements, test repositories, and historical defects;
6. A careful CI/CD hook for test execution, and session history that survives a restart and can be audited; and
7. Load, reliability, accessibility, privacy, and security reviews before this is treated as a wider deployment.

References

[1] International Organization for Standardization. *ISO/IEC/IEEE 29119-1:2022: Software and systems engineering—Software testing—Part 1: General concepts*. ISO, 2022. 1, 6

[2] International Software Testing Qualifications Board. Certified tester foundation level syllabus, version 4.0. https://www.istqb.org/certifications/certified-tester-foundation-level, 2023. 1, 6

[3] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024. doi: 10.1109/TSE.2024.3368208. 1, 8

[4] Monika Tyagi. Quality assistance application: Project source code repository. https://github.com/monikatyagiisc/quality-assistance-app, 2026. Accessed November 2026. 4, 22

[5] Katalon. Katalon true: The AI platform for software quality. https://katalon.com/. Accessed November 2026. 7

[6] testRigor. testrigor: Generative AI-based test automation tool. https://testrigor.com/. Accessed November 2026. 7

[7] Diffblue. Diffblue: Automated unit test generation. https://www.diffblue.com/. Accessed November 2026. 7

REFERENCES

[8] Tonic.ai. Tonic.ai: Synthetic data platform. https://www.tonic.ai/. Accessed November 2026. 7

[9] mabl. mabl: Agentic testing platform. https://www.mabl.com/. Accessed November 2026. 7

[10] Applitools. Applitools visual AI testing platform. https://applitools.com/. Accessed November 2026. 7

[11] Saleema Amershi, Andrew Begel, Christian Bird, Robert De Line, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice*, pages 291–300, 2019. doi: 10.1109/ICSE-SEIP.2019.00042. 8

[12] Aazaade Faraji and Nuno Pombo. AI-driven software test automation: An AI4SE-oriented survey of techniques, tools, and challenges. *IEEE Access*, 13:183296–183313, 2025. doi: 10.1109/ACCESS.2025.3623944. 8

[13] Afonso Fontes and Gregory Gay. The integration of machine learning into automated test generation: A systematic mapping study. *Software Testing, Verification and Reliability*, 33(4):e1845, 2023. doi: 10.1002/stvr.1845. 8

[14] Navid Bin Hasan, Md. Ashraful Islam, Junaed Younus Khan, Sanjida Senjik, and Anindya Iqbal. Automatic high-level test case generation using large language models. In *International Conference on Mining Software Repositories (MSR)*, 2025. arXiv:2503.17998. 8

[15] Mariam Lahami and Moez Krichen. Architectural perspectives on GenAI-driven software testing systems: A systematic literature review. *Information and Software Technology*, 199:108246, 2026. doi: 10.1016/j.infsof.2026.108246. 9

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. 9

REFERENCES

[17] Lei Wang, Cheng Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wei Zhao, Zhe Wei, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024. 9

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. 9

[19] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023. 10

[20] Sebastián Ramírez. Fastapi documentation. https://fastapi.tiangolo.com/. Accessed November 2026. 17

[21] Google. Agent development kit documentation. https://google.github.io/adk-docs/. Accessed November 2026. 18

[22] Model Context Protocol. What is the model context protocol (MCP)? https://modelcontextprotocol.io/introduction. Accessed November 2026. 20